Machia — System Architecture

> **An agent runtime for N-ATLAS.** Give it a task in English, Hausa, Igbo or Yoruba; it plans with N-ATLAS, calls tools to act on your PC, and writes itself new skills so it gets better over time.

Status: v0 design (straw man — open decisions flagged in §12) **Target:** NAIC 2026 · Problem Statement 1 — *Developer Infrastructure* · submission 12 Oct 2026, 11:59 PM WAT

---

1. The one-line pitch

N-ATLAS ships with tool-calling in its chat template but **no agent runtime**. Machia is that missing layer: a local-first Python agent framework where N-ATLAS is the brain, the PC is the hands, and `SKILL.md` files are the long-term memory — plus a self-improvement loop where the agent writes its own skills.

2. Hackathon constraints we are designing against (verified)

Fact	Source	Design consequence
Build-only; a working artefact is mandatory	naic page	Ship a runnable app, not a framework pitch
PS1 needs ≥ 2 external beta testers	naic page	Need a 15-minute install + quickstart per tester
Deadline **12 Oct 2026, 11:59 PM WAT**	naic page	8 days from 4 Oct
Wrapping a general-purpose model (GPT-4 etc.) = **disqualified**	FAQ	N-ATLAS must be the only model on every code path used in the demo — no silent fallback
7 submission components (artefact, integration evidence, validation, docs, 3–5 min video, team, endorsement)	naic page	Docs are a *graded artefact*, not an afterthought
Eval = Working Artefact & Technical Rigour, **N-ATLAS Integration Depth**, Real-World Validation, Impact, Scalability, Team	naic page	Constrained tool-calling + N-ATLAS-native prompt format is the integration story
N-ATLAS = Llama-3 8B fine-tune; Yoruba, Hausa, Igbo, Nigerian English + ASR	naic page	Free multilingual + voice angle; no vision input

3. N-ATLAS capabilities we verified at the model level

Pulled from the published chat template and config (community mirrors; official repo is gated):

Property	Value	Why it matters
Base	`LlamaForCausalLM`, 8B, 32 layers, vocab 128,256	Standard llama.cpp / vLLM serving
Native tool calling	Chat template accepts `tools=` and instructs: "*Respond in the format `{"name": function name, "parameters": dictionary of argument name and its value}`*"	**We get real tool calls — no fragile ReAct text scraping needed**
Tool-call cardinality	Template raises `"This model only supports single tool-calls at once!"`	Agent loop must be strictly **one tool per turn** — no parallel calls
Tool-result role	`tool` / `ipython`	Llama-3.1-style tool protocol
Stop tokens	eos `128001`, `128008` (`<|eom_id|>`), `128009` (`<|eot_id|>`)	Must set stop sequences or the loop rambles
System preamble	Auto-injects `Cutting Knowledge Date: December 2023` and `Today Date: <date>`	We can inject the real current date for a PC agent
Context	Card states **8,092 tokens**; config declares RoPE scaling (factor 8, `original_max_position_embeddings` 8192)	**Budget the prompt at ~8K.** Progressive disclosure of skills is mandatory, not optional
Weights access	`NCAIR1/N-ATLaS` is **gated** (auth required); `QuantFactory/N-ATLaS-GGUF` is **public** (Q2_K → Q8)	Request official access Day 1; start building today on the public Q4_K_M quant

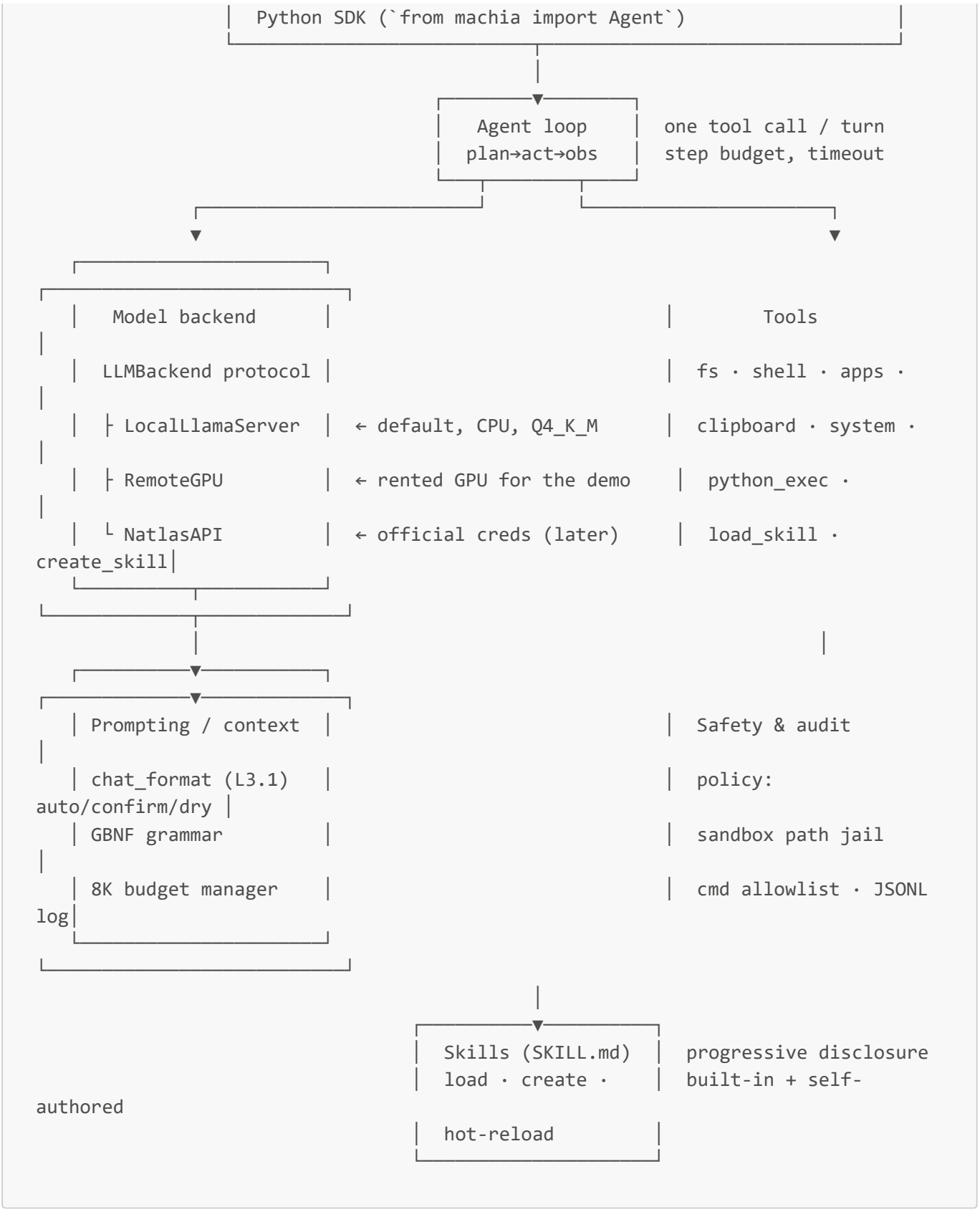
4. Build machine reality (verified)

```
GPU          none (nvidia-smi not found)  → CPU-only inference
CPU / RAM    8 cores / 34 GB
Disk         18 GB free on C:             → Q4_K_M ≈ 4.9 GB, fits
Toolchain    Python 3.11 + uv, Node 26, npm 11, Docker 29, git, ripgrep
Missing      ollama, HF token (gated download needs one)
```

Expected local throughput for an 8B Q4 model on 8 CPU cores: **roughly 4–8 tok/s**. A 5-step task therefore takes ~1–3 minutes. That is fine for the submission video and for correctness testing, and it is why §5 has a swappable backend.

5. Architecture

```
                    ┌──────────────────────── interfaces ────────────────────────┐
                    │  CLI REPL        FastAPI + SSE        Next.js web UI       │
```



5.1 Model backend — one interface, three implementations

```
class LLMBackend(Protocol):
    def chat(self, messages: list[Message], tools: list[dict] | None = None,
             grammar: str | None = None, stop: list[str] | None = None) ->
```

```
LLMResponse: ...
    def count_tokens(self, text: str) -> int: ...
```

- `LocalLlamaServer` (default) — talks HTTP to a `llama.cpp` `llama-server` process serving `N-ATLaS.Q4_K_M.gguf` with `--jinja` (so the model's own chat template is used, exactly as published) and grammar support. **Chosen over `llama-cpp-python` to avoid Windows C++ build pain.**
- `RemoteGPU` — same HTTP shape pointed at a rented GPU (vLLM or llama-server) with the official weights at bf16. Used to record the demo video at ~40–80 tok/s and later to validate against the real model.
- `NatlasAPI` — the official N-ATLAS API surface, wired once shortlisted teams receive credentials.

The SDK exposes `--backend` so a developer can swap without touching agent code. Every backend speaks the **N-ATLAS chat template**; there is no path in the demo that calls a non-N-ATLAS model.

5.2 The agent loop (N-ATLAS-shaped)

N-ATLAS emits exactly one tool call per turn, so the loop is deliberately simple:

1. Build the system prompt: identity, sandbox root, OS info, **skill index (name + one-line description only)**.
2. Build the user turn with the tool schema block in the N-ATLAS format.
3. Generate with a **GBNF grammar generated from the tool JSON schemas** → the model *cannot* emit malformed JSON, an unknown tool name, or an argument with a wrong type. This is the core reliability trick, and it is itself a reusable contribution for anyone building on N-ATLAS.
4. Parse `{"name", "parameters"}` → validate with Pydantic → check safety policy.
5. Execute in the sandbox; append the result as a `tool` message.
6. Repeat until the model calls `finish`, or the step/token budget is exhausted.
7. Write a run trace (JSONL): steps, tool calls, latencies, tokens, failures.

Fallbacks when the grammar path is unavailable (e.g. official API without grammar support): a schema-pinning prompt, name whitelisting, and a single repair retry that re-asks with the parse error. Never a second model.

5.3 Tool layer — PC actuation with a risk rating

Every tool is a Python function plus a `ToolSpec` carrying its JSON schema and safety metadata. The registry auto-generates both the prompt block and the GBNF grammar.

Tool	Risk	Notes
`fs_list`, `fs_read`	low	read-only, path-jailed to the workspace root
`fs_write`, `fs_move`, `fs_delete`	medium	path-jailed; `delete` always confirmed
`shell_exec`	**high**	command allowlist + deny-list, confirm-by-default, timeout, no shell interpolation
`python_exec`	high	subprocess in the sandbox root, confirm-by-default
`open_app`, `open_url`, `clipboard_read/write`	medium	clipboard write is the classic "exfiltration" surface — logged

Tool	Risk	Notes
`sys_info`	low	CPU/RAM/disk/OS — also grounds the model in reality
`ask_user`	—	agent asks the human instead of guessing
`finish`	—	explicit clean exit; the loop terminates only on this or budget

Safety modes: `dry-run` (plan only, no side effects), `confirm` (default — every medium/high tool stops for a y/n, surfaced in the web UI), `auto` (allowlist-only, for the scripted demo). Everything is appended to `~/.machia/audit.jsonl`.

Deliberate non-goal: screenshot-driven GUI control. N-ATLAS has ASR but **no vision**, so there is no VLM to interpret a screenshot. GUI work, if any, would go through the Windows accessibility tree (`pywinauto`) — a stretch goal, not part of the critical path.

5.4 Skill layer — the differentiator

A skill is a directory with a markdown file, Hermes-style:

```
---
name: organize-downloads
description: Sort files in the Downloads folder into subfolders by type.
when_to_use: User asks to clean up, sort or organize downloads or a messy folder.
version: 1
tools: [fs_list, fs_move]
---

## Procedure
1. `fs_list` the target folder...
```

Progressive disclosure is a hard requirement (8K context): only `name` + `description` live in the system prompt; the body is pulled in on demand via the `load_skill` tool. With ~10 built-in skills that costs ~200 tokens of prompt instead of ~4,000.

Self-improvement loop — the demo centrepiece:

```
task fails / repeats  →  agent drafts a SKILL.md via create_skill
                      →  frontmatter schema-validated, python body syntax-checked
                      →  written to skills/library/ and hot-reloaded into the
registry
                      →  available in the very next session, in any language
```

Executable skills (`create_skill` with a `tool.py`) require explicit user approval — that is arbitrary code execution, so it is gated the same way as `shell_exec`.

5.5 Interfaces

- **Python SDK** — `Agent(model=..., skills_dir=...).run("Tidy my downloads")`. This is the literal PS1 deliverable: a clean abstraction over N-ATLAS that another Nigerian developer can adopt.
- **CLI REPL** — for devs and for the live demo.
- **FastAPI + SSE** — streams tokens, tool calls and approval prompts.
- **Next.js web UI** — chat pane, live tool-call trace, skill browser, approval dialog. Streams the agent's reasoning about what it is doing to the user's computer.

5.6 Evaluation & beta-tester harness

- `evals/tasks/*.yaml` — 30–50 concrete tasks (single-step file ops → multi-step shell + skill reuse), scored pass/fail and by token/latency cost, runnable against any backend.
- `scripts/beta_pack/` — one-command install, 5 sample tasks, a feedback form, and a script that collects the trace + a short questionnaire. **This is what produces the ≥ 2 documented external beta-test runs PS1 requires** — build it on day 5, hand it to testers on day 7, not the night before.

6. Repo layout

```
machia-agent/
├─ docs/           ARCHITECTURE.md · TOOL_PROTOCOL.md · SKILL_FORMAT.md · NAIC_COMPLIANCE.md
├─ src/machia/
│  ├─ agent.py     ← the loop        config.py   trace.py
│  ├─ backends/    base.py · llamacpp.py · remote.py · natlas_api.py
│  ├─ prompting/   chat_format.py · grammar.py · context.py
│  ├─ tools/       registry.py · spec.py · fs.py · shell.py · apps.py · meta.py
│  ├─ skills/      loader.py · creator.py · library/
│  ├─ safety/      policy.py · sandbox.py · audit.py
│  └─ server/      app.py
├─ web/            Next.js UI
├─ evals/          tasks/ · run_eval.py
├─ scripts/        setup_models · run_llama_server · beta_pack/
└─ tests/
```

7. Build roadmap (4 Oct → 12 Oct)

Day	Deliverable	Gate
1 · Oct 4–5	Scaffold, download Q4_K_M, `llama-server` up, `LocalLlamaServer`, **tool-call spike**	A real `{"name","parameters"}` call comes back from N-ATLAS
2 · Oct 6	Agent loop, 6 core tools, sandbox + policy + audit trail	Agent lists and reorganises a test folder end-to-end
3 · Oct 7	Skill layer: loader, progressive disclosure, `load_skill`, `create_skill`	Agent writes a skill, next session reuses it

Day	Deliverable	Gate
4 · Oct 8	FastAPI + SSE + Next.js UI, approval dialogs	Live demo works in the browser
5 · Oct 9	Eval harness, GBNF grammar path, hardening	30 tasks run, pass-rate recorded
6 · Oct 10	Docs (EN + pidgin/Yoruba quickstart), deployment, beta pack	A stranger can install from the README
7 · Oct 11	**External beta testers run it**, evidence collected	≥ 2 documented tester runs with traces
8 · Oct 12	3–5 min video, submission package, submit	Submitted before 11:59 PM WAT

8. Risks and mitigations

Risk	Severity	Mitigation
No GPU → 4–8 tok/s locally	High	Local backend for correctness; rent a GPU by the hour (pay-as-you-go, no upfront minimum) to record the video
Official `NCAIR1/N-ATLaS` weights are gated	High	Request access on day 1; build against the public Q4_K_M quant in parallel; ship official-weights instructions and validate against bf16 before submitting
8K context is easily blown by tool schemas + history	High	Grammar + skill index are generated compactly; budget manager trims tool results; history summarisation after N turns
8B model fumbles multi-step plans	Medium	One-tool-per-turn loop, `ask_user` escape hatch, narrow task suite in the demo, step budget
"Wrapping another model" disqualification	**Critical**	No non-N-ATLAS fallback on any demo path; the trace log records the model id of every call and is submitted as integration evidence
Windows shell differences (bash vs cmd)	Medium	`shell_exec` is shell-agnostic and configurable; assert behaviour in tests on this machine
18 GB free disk	Medium	Q4_K_M is 4.9 GB; keep only one quant; clean up before adding a second
Solo build, 8 days	High	Scope is fixed at the day-4 UI; voice and GUI automation are explicitly stretch-only

9. Positioning against the other PS1 entries (expected field)

Most teams will build an SDK, a playground, or fine-tuning scripts. The unclaimed high-value slot is **"N-ATLAS can call tools, but nobody has made it *do* things."** Constrained, grammar-backed tool calling plus a self-writing skill library is a substantially deeper integration than a chat playground, and it gives "Impact Potential"

a concrete story: *a Nigerian developer can build an offline, sovereign, multilingual agent on a cheap laptop, with no cloud dependency.*

10. Multilingual & voice angle

Because the language capability is free (same weights), we get a disproportionate impact story:

- Accept the task in English, Hausa, Igbo, Yoruba or Nigerian Pidgin; reply in the same language.
- Skills' `description`/`when_to_use` stay English for retrieval reliability; narration is localised.
- **Stretch:** voice control via the official N-ATLAS ASR — speak a task in Hausa, the agent does it. Access to the ASR service may only come with accreditation, so it is scoped as optional.

11. What "done" looks like

A user types (or says) "*my desktop is a mess, sort it out*" → N-ATLAS asks one clarifying question → plans → moves files (each risky step surfaced for approval) → notices the same pattern it handled yesterday → offers to save it as a skill → the skill appears in `skills/library/` and is reusable next session in any supported language. A 3–5 minute video of that, a public repo, an eval table, and two beta testers' traces = a defensible PS1 submission.

12. Open decisions

#	Decision	Options	Recommendation
D1	Problem statement	PS1 Developer Infrastructure · PS2 Voice-First · PS3 Sectoral Fine-Tuning	**PS1** — the agent runtime is infrastructure, and its validation bar (2 beta testers) is the only one achievable solo in 8 days, versus PS2's 50 documented user interactions or PS3's 500-example benchmark
D2	Serving	local CPU only · cloud GPU only · **both behind one adapter**	**Both.** Local proves honesty and offline operation; the rental exists to make the video watchable
D3	How far into the PC	files only · **files + shell + apps** · + GUI automation	**files + shell + apps**, with policy gates. GUI is a stretch goal with no VLM to drive it
D4	Team & track	solo independent dev (Track B) · with a team	Confirm Track B eligibility docs (valid ID or CAC) and line up the 2 beta testers early
D5	Product name	`machia` (current folder name) · a Nigerian-flavoured name	Name is cheap to change; decide before docs and video